

Maffe – A Lagrangian Approach to Maximum Agreement Forests

Anton Hoof ✉

Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany

Tobias Röhr ✉

Hasso Plattner Institute, University of Potsdam, Germany

Abstract

We describe Maffe, a solver for the maximum agreement forest (MAF) problem on rooted phylogenetic trees, submitted to the 2026 PACE challenge. Maffe combines reduction rules, a branch-and-price exact solver with a Lagrangian pricer, and a Lagrangian relaxation-based heuristic for larger instances.

2012 ACM Subject Classification Theory of computation → Packing and covering problems; Theory of computation → Integer programming; Mathematics of computing → Combinatorial optimization; Applied computing → Operations research

Keywords and phrases maximum agreement forest, phylogenetic trees, integer programming, branch-and-price, column generation, Lagrangian relaxation, heuristic algorithms

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Acknowledgements The implementation was assisted by OpenAI's Codex with GPT 5.4 and GPT 5.5. We extensively tested the solver with fuzzing and STRIDE instances and take responsibility for its correctness.

1 Preliminaries and Notation

Let $T_1, \dots, T_k$ be rooted phylogenetic trees on a common leaf set $\mathcal{L}$, and let $n = |\mathcal{L}|$. For a tree T , we write $V(T)$ and $E(T)$ for its vertex and edge sets, and use $V(T) \setminus \mathcal{L}$ for its internal vertices. For a non-empty vertex set $X \subseteq V(T)$, $\text{lca}(T, X)$ denotes the least common ancestor of X in T .

For $X \subseteq V(T)$, the Steiner tree $\text{ST}(T, X)$ is the unique minimal connected subgraph of T containing all vertices in X . When the context is clear, we also write $v \in \text{ST}(T, X)$ to mean that v is a vertex of this subgraph, and $e \in E(\text{ST}(T, X))$ to mean that e is one of its edges. A non-empty leaf set $A \subseteq \mathcal{L}$ is *compatible* if the restrictions of $T_1, \dots, T_k$ to A have the same rooted topology. We write $\mathcal{C}$ for the family of all compatible leaf sets, and $\mathcal{C}' = \{A \in \mathcal{C} \mid |A| \geq 2\}$ for the non-singleton compatible sets. A partition $P \subseteq \mathcal{C}$ is a family of pairwise disjoint compatible sets whose union is $\mathcal{L}$. Such a partition is an agreement forest if, for every input tree T_i , the Steiner trees $\text{ST}(T_i, A)$ with $A \in P$ are pairwise disjoint on internal vertices. Equivalently, each block of P can be separated from the other blocks by cutting edges in every input tree.

The maximum agreement forest (MAF) problem asks for an agreement forest with the minimum number of components.

2 Reduction Rules

We use reductions that either replace a fixed part of the final forest by a smaller representative, mark edges that must be cut, or split the instance into subinstances whose solutions can be lifted and combined. The multi-tree setting follows Shi et al. [8]; the common subtree and common chain reductions, including the truncation length for multiple binary trees, follow



© Anton Hoof and Tobias Röhr;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:9

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

42 Kelk et al. [4]. Cluster partitioning follows Li and Zeh [5], and the two-tree 3-2 rule is based
43 on the rooted kernel of Kelk, Linz, and Meuwese [3].

- 44 ■ **Cherry reduction.** A common pendant binary subtree is compressed to a representative
45 leaf, which is expanded again when the reduced solution is lifted.
- 46 ■ **Chain reduction.** A common chain is shortened to $k + 1$ leaves, where k is the number
47 of input trees.
- 48 ■ **Cluster reduction.** Any common cluster, or its complement, can be separated from
49 the rest of the instance. We search for a smallest such cluster. The smaller side is solved
50 twice, once with a dummy leaf representing the rest of the instance and once without
51 this dummy leaf. If the dummy-leaf solution is better, the remaining instance receives a
52 leaf representing the solved cluster; otherwise nothing is attached.
- 53 ■ **3-2 reduction.** For two-tree instances, a cherry in one tree whose two leaves have exactly
54 one hanging subtree attached to their connecting path in the other tree forces the edge
55 to that hanging subtree to be cut.
- 56 ■ **5-3 reduction.** The implementation also uses the analogous triple case: if a rooted
57 triple in one tree forces two hanging subtrees in the other tree to be separated, both
58 corresponding edges are marked as cut.

59 The cluster reduction is also definable once known cut edges have turned the input trees
60 into forests. In this setting, a leaf set S is accepted as a cluster if, in each forest, the leaves
61 of S are covered by whole connected components of the forest together with at most one
62 component that contains both leaves from S and leaves from $\mathcal{L} \setminus S$. That partial component
63 plays the role of the ordinary cluster subtree; all other components used by S are already
64 separated by known cuts. This forest version reduced several larger heuristic instances.

65 3 Two-Tree Branch-and-Price

66 All our two-tree solvers use a modified version of the exponential-size ILP formulation of
67 Olver et al. [7].

68 For every non-singleton compatible set $A \in \mathcal{C}'$, we introduce a binary variable x_A
69 indicating whether A is selected as a component of the forest. Singleton components are
70 implicit: any leaf not covered by a selected non-singleton component becomes a singleton
71 component. If a selected component A has size $|A|$, then it replaces $|A|$ singleton components
72 by one component and therefore saves $|A| - 1$ components. Minimizing the final number of
73 components is thus equivalent to the following weighted set packing problem:

$$\begin{aligned}
74 \quad \min \quad & n - \sum_{A \in \mathcal{C}'} x_A (|A| - 1) \\
75 \quad \text{s.t.} \quad & \sum_{A \in \mathcal{C}': v \in \text{ST}(T_1, A)} x_A \leq 1 & \forall v \in V(T_1) \setminus \mathcal{L}, & \text{(P1)} \\
76 \quad & \sum_{A \in \mathcal{C}': v \in \text{ST}(T_2, A)} x_A \leq 1 & \forall v \in V(T_2) \setminus \mathcal{L}, & \text{(P2)} \\
77 \quad & x_A \in \{0, 1\} & \forall A \in \mathcal{C}'. &
\end{aligned}$$

78 The packing constraints P1 and P2 enforce that selected components have internally disjoint
79 Steiner trees in both T_1 and T_2 . They also imply that a leaf is covered by at most one
80 selected non-singleton component: if $\ell \in A$ and $|A| \geq 2$, then the parent of ℓ in T_1 and in T_2
81 lies in the corresponding Steiner tree.

The ILP formulation has a tight LP relaxation in practice, as observed by Olver et al. [7] and Frohn et al. for the unrooted version [1]. Since there are exponentially many variables, we solve the LP relaxation using column generation. Frohn et al. [1] solve the pricing problem by finding a single component with the most negative reduced cost.

This is equivalent to the rooted weighted maximum agreement subtree (rWMAST) problem, where one asks for a compatible leaf set of maximum total weight. For two trees, rWMAST can be solved by standard dynamic programming in $\mathcal{O}(n^2)$ time [1]. In our experiments, the rWMAST pricer led to slow LP convergence because it returns only one component per iteration and the LP is highly degenerate. Instead, Maffe uses the Lagrangian dual as a pricer, which can return several negative reduced cost components in one iteration.

3.1 Lagrangian Relaxation

We dualize the set packing constraint P2 associated with the internal vertices of T_2 . The packing constraints of T_1 remain part of the feasible domain of the Lagrangian subproblem. This yields the Lagrangian functional

$$\begin{aligned}
 F(\lambda) = \min_x \quad & n - \sum_{A \in \mathcal{C}'} x_A (|A| - 1) \\
 & + \sum_{v \in V(T_2) \setminus \mathcal{L}} \lambda_v \left(-1 + \sum_{A \in \mathcal{C}': v \in \text{ST}(T_2, A)} x_A \right) \\
 \text{subject to} \quad & \sum_{A \in \mathcal{C}': v \in \text{ST}(T_1, A)} x_A \leq 1 \quad \forall v \in V(T_1) \setminus \mathcal{L}, \\
 & x_A \in \{0, 1\} \quad \forall A \in \mathcal{C}',
 \end{aligned}$$

with Lagrangian master $\max_{\lambda \geq 0} F(\lambda)$.

For fixed multipliers, optimizing over x is equivalent to selecting compatible components whose Steiner trees on T_1 do not intersect. The contribution of one non-singleton component A is

$$w_\lambda(A) = |A| - 1 - \sum_{v \in \text{ST}(T_2, A) \setminus \mathcal{L}} \lambda_v.$$

Since $\sum_{A \in P} |A| = n$ for every leaf partition P , the cardinality term is fixed. After changing signs, evaluating the Lagrangian subproblem becomes a maximum-weight partition problem with a penalty of -1 per component. The resulting subproblem is the rooted weighted maximum agreement subtree partition (rWMASTP) problem. We describe a dynamic programming solution for two trees in time $\mathcal{O}(n^2)$ in Appendix A.

3.2 Branching and Pricing

To keep our branching decisions compatible with the pricing subproblem, we branch on edges of the input trees: whether they are part of a component (forced) or not (cut). For a current fractional LP solution x^* , define the usage of an edge $e \in E(T_j)$ as $y_e^j = \sum_{A \in \mathcal{C}': e \in E(\text{ST}(T_j, A))} x_A^*$. In any integral agreement forest, $y_e^j \in \{0, 1\}$. If $0 < y_e^j < 1$, we create two children: one where e is cut and one where e is forced to be used by exactly one selected component.

Let $E_{\text{cut}}^1 \subseteq E(T_1)$ and $E_{\text{cut}}^2 \subseteq E(T_2)$ be the cut edges, and let $E_{\text{forced}}^1 \subseteq E(T_1)$ and $E_{\text{forced}}^2 \subseteq E(T_2)$ be the forced edges, which arise from reductions and branching decisions.

119 To solve the LP relaxation of the ILP formulation using column generation, we must find
 120 compatible components $A \in \mathcal{C}'$ whose corresponding variables x_A have negative reduced
 121 costs w.r.t. the given LP duals. For LP duals w_v^1 and w_v^2 for the packing constraints (P1)
 122 and (P2), respectively, and w_e^1 and w_e^2 for the forced edge constraints

$$\begin{aligned}
 123 \quad & \sum_{A \in \mathcal{C}' : e \in E(\text{ST}(T_1, A))} x_A = 1 & \forall e \in E_{\text{forced}}^1, \\
 124 \quad & \sum_{A \in \mathcal{C}' : e \in E(\text{ST}(T_2, A))} x_A = 1 & \forall e \in E_{\text{forced}}^2, \tag{F}
 \end{aligned}$$

125 the reduced cost $\bar{c}_A$ of a variable x_A for a component $A \in \mathcal{C}'$ arises from the LP relaxation
 126 as:

$$\begin{aligned}
 127 \quad \bar{c}_A = & 1 - |A| + \sum_{v \in \text{ST}(T_1, A) \setminus \mathcal{L}} w_v^1 + \sum_{v \in \text{ST}(T_2, A) \setminus \mathcal{L}} w_v^2 \\
 128 \quad & + \sum_{e \in E(\text{ST}(T_1, A)) \cap E_{\text{forced}}^1} w_e^1 + \sum_{e \in E(\text{ST}(T_2, A)) \cap E_{\text{forced}}^2} w_e^2.
 \end{aligned}$$

129 Here $w_v^1, w_v^2 \geq 0$ are the dual variables associated with the packing constraints (P1) and (P2),
 130 and w_e^1, w_e^2 are the dual variables associated with the forced edge constraints (F). The pricing
 131 subproblem then consists of finding a compatible component $A \in \mathcal{C}'$ with negative reduced
 132 cost:

$$\begin{aligned}
 133 \quad & \text{find } A \in \mathcal{C}' \\
 134 \quad & \text{s.t. } \bar{c}_A < 0, \\
 135 \quad & E(\text{ST}(T_1, A)) \cap E_{\text{cut}}^1 = \emptyset, \\
 136 \quad & E(\text{ST}(T_2, A)) \cap E_{\text{cut}}^2 = \emptyset.
 \end{aligned}$$

137 While Frohn et al. [1] solve rWMAST to find a single column with the most negative reduced
 138 cost, we reduce the pricing problem to rWMASTP to generate multiple negative reduced
 139 cost columns in a single pricing iteration.

140 Let $P \subseteq \mathcal{C}$ be a leaf partition returned by rWMASTP with weights induced by the
 141 negative vertex and edge duals above. If there exists a compatible component $A \in \mathcal{C}'$ with
 142 negative reduced cost, then at least one non-singleton component in P also has negative
 143 reduced cost. To see this, note that the total weight of any leaf partition P is

$$144 \quad \sum_{A \in P} w(A) = -n - \sum_{A \in P \cap \mathcal{C}'} \bar{c}_A,$$

145 where the baseline partition consisting only of singleton leaves has weight $-n$. A partition
 146 containing only components with non-negative reduced costs has total weight at most $-n$.
 147 Conversely, if there exists a component $A \in \mathcal{C}'$ with $\bar{c}_A < 0$, the partition consisting of A
 148 and the remaining leaves as singletons has total weight strictly greater than $-n$. It follows
 149 that the optimal partition returned by rWMASTP must also have weight greater than $-n$,
 150 which is possible only if it contains a non-singleton component with negative reduced cost.

151 **4 Multiple Tree Compact ILP**

152 The set-packing formulation above also extends to $k \geq 3$ trees by adding packing constraints
 153 for every tree. Eligible reduced multi-tree instances are therefore solved with the compact

ILP instead of branch-and-price. SCIP can apply its general MIP cuts to the compact model, which strengthens the LP relaxation. For this back end, we use the compact ILP formulation that is equivalent to the set-packing formulation given in [7]. Note that while Olver et al. describe the compact ILP only for two trees, the same construction extends to k trees:

4.1 Formulation

Assume that the leaves are numbered $1, \dots, n$. Let $\mathcal{Z} = \{(a, b) \mid 1 \leq a \leq b \leq n\}$ be the set of states and let $\mathcal{Z}_{\mathcal{L}} = \{(a, a) \mid a \in \mathcal{L}\}$ be the leaf states. For a state $r = (a, b)$, write $m_t(r) = \text{lca}(T_t, \{a, b\})$. We construct a directed acyclic graph with left arcs U_L and right arcs U_R . For states $r = (a, b)$ and $s = (c, d)$, the arc (r, s) is present if $m_t(s)$ is a strict descendant of $m_t(r)$ in every tree T_t ; it is a left arc if $a = c$, and a right arc if $b = c$. Let $U = U_L \cup U_R$. For a state r , write $L(r) = \delta^+(r) \cap U_L$, $R(r) = \delta^+(r) \cap U_R$, and $I(r) = \delta^-(r)$. Write $[k] = \{1, \dots, k\}$ and $\mathcal{I}_t = V(T_t) \setminus \mathcal{L}$. For a set A of arcs, write $y(A) = \sum_{a \in A} y_a$.

Using the convention of Olver et al., the objective value below is the number of components minus one. The compact LP relaxation is

$$\begin{aligned}
 \min \quad & \sum_{r \in \mathcal{Z} \setminus \mathcal{Z}_{\mathcal{L}}} (y(L(r)) - y(I(r))) + \sum_{\ell \in \mathcal{L}} x_{\{\ell\}} - 1 \\
 \text{s.t.} \quad & y(L(r)) = y(R(r)) & \forall r \in \mathcal{Z} \setminus \mathcal{Z}_{\mathcal{L}}, & \text{(balance)} \\
 & y(L(r)) \geq y(I(r)) & \forall r \in \mathcal{Z} \setminus \mathcal{Z}_{\mathcal{L}}, & \text{(flow)} \\
 & x_{\{\ell\}} = 1 - y(I((\ell, \ell))) & \forall \ell \in \mathcal{L}, & \text{(leaf)} \\
 & \sum_{a \in \Gamma_t(v)} y_a \leq 1 & \forall t \in [k], v \in \mathcal{I}_t, & \text{(capacity)} \\
 & x_{\{\ell\}} \geq 0 & \forall \ell \in \mathcal{L}, & \text{(nonneg-}x\text{)} \\
 & y_a \geq 0 & \forall a \in U. & \text{(nonneg-}y\text{)}
 \end{aligned}$$

Here $\Gamma_t(v)$ is the set of compact arcs whose corresponding transition uses the internal vertex v in T_t . More explicitly, for an arc $a = (r, s)$, the transition uses the vertices on the path from $m_t(r)$ to $m_t(s)$, counting $m_t(r)$ only for left arcs and not counting $m_t(s)$ if s is an internal state. The fourth constraint is the additional capacity constraint missing from the printed compact formulation of Olver et al.; without it, the compact flow can select components whose Steiner trees overlap away from their roots. The compact ILP is obtained by requiring the arc variables y_a and singleton variables $x_{\{\ell\}}$ to be integral.

The compact state space gives cuts for the multi-tree exponential formulation. Some valid inequalities over compact arcs lift to cuts over compatible components; if the pricer can evaluate the resulting component coefficient, these cuts remain compatible with column generation.

4.2 Model Size

The formulation has $\mathcal{O}(n^2)$ states and $\mathcal{O}(n^3)$ compact arcs, and therefore already contains $\mathcal{O}(n^3)$ transition variables before the capacity constraints are considered. This is polynomial, but still too large to build in full for the instances targeted by the compact solver.

4.3 Model Size Reductions

The implementation reduces the model size in three ways. First, the capacity constraints are handled lazily: the solver starts without all rows of type (capacity), computes the capacity

usage of the current solution, and adds only violated capacity rows. Before the compact ILP is solved, we run the root relaxation of the branch-and-price model by column generation. The resulting columns are used to compute an initial incumbent for the compact model. We also add the capacity rows that are needed for this LP certificate, namely rows with relevant dual value or support in the root relaxation, before the lazy capacity separator is enabled.

Second, equivalent states are merged. The mathematical description above indexes states by leaf pairs. In the implementation, two states r and s are identified if

$$(m_1(r), \dots, m_k(r)) = (m_1(s), \dots, m_k(s)).$$

All tests for valid transitions, all flow coefficients, and all capacity usages depend on a state only through this tuple of least common ancestors. Identifying such states therefore only removes duplicate copies of the same compact-flow state. After the state merge, parallel transition arcs with the same source, target, and orientation are also represented by a single variable.

Third, we fold transition bicliques. For one fixed orientation, suppose the transition graph contains all arcs $S \times Q$ between a set S of source states and a set Q of target states. Choosing arcs inside this complete bipartite graph only has to decide which sources are used and how many arcs enter each target. The implementation therefore replaces the variables $y_{s,q}$ by source variables $p_s \in \{0, 1\}$ for $s \in S$ and target-count variables $z_q \in \mathbb{Z}_{\geq 0}$ for $q \in Q$, together with the row $\sum_{s \in S} p_s = \sum_{q \in Q} z_q$. The source-side objective and flow coefficients are assigned to the variables p_s , while the target-side flow coefficients are assigned to the variables z_q . Bicliques are folded only when every affected capacity row can be charged consistently to one of these two sides. Any original integral solution projects to these counts, and any integral folded solution can be expanded by matching selected sources to target counts inside $S \times Q$. This preserves the objective value and every retained or lazily generated capacity row.

5 Heuristic Solver

The heuristic solver first applies the same reductions as the exact solver, but the exact solves used inside cluster reductions have a time limit and use the best incumbent found within that limit. After reductions, the remaining instance size determines the next step. For small and medium-size instances, the B&P solver is run and returns the best incumbent if the time limit is reached. For larger instances, solving the full LP is too expensive, so we use a two-stage set-packing heuristic.

The first stage generates candidate components. We optimize the Lagrangian dual from Section 3.1 using a bundle method [2] and record the components found while evaluating the pricing subproblems. Since the generated components become less diverse once the dual values stabilize, we also evaluate the Lagrangian functional at random perturbations of the best dual values found so far. The same process is run once with the packing constraints of T_2 dualized and once with the roles of T_1 and T_2 exchanged.

The second stage solves restricted set-packing ILPs over the generated components. Components are ranked by how often they were generated, with component size and a random value used as tiebreakers. The heuristic then considers increasingly larger prefixes of this ranking. If time remains after the final restricted ILP, the incumbent is used as a warm start for a final B&P fallback.

6 Implementation Details

The implementation uses SCIP Optimization Suite 10.0.2 as MIP framework. SCIP uses HiGHS 1.15.1 as LPI backend.

6.1 Exact Track

After reductions, the exact track chooses between exact back ends according to the residual instance. Very small residual instances are solved directly. Multi-tree residual instances with $k \geq 3$, fewer than 500 leaves, and no fixed edge states use the compact ILP formulation from Section 4. All remaining residual instances use a SCIP branch-and-price algorithm over the set-packing formulation.

In the branch-and-price back end, each search node maintains the cut and forced edge sets introduced above. At a node, the restricted LP is solved, the rWMASTP pricer is called with the current dual values, and all generated components with negative reduced cost are added as columns. If the restricted LP is infeasible, Farkas pricing with the corresponding Farkas duals generates columns that restore feasibility. The pricing loop terminates when the pricer certifies that no improving component remains for the current node.

Branching decisions fix edges either as cut edges or as forced edges. These decisions are propagated to the pricer by forbidding transitions through cut edges and by adding the forced-edge constraints (F). The solver disables SCIP's default separation and instead relies on the structure of the packing formulation, the custom branching decisions, and a sub-MIP heuristic over the current column pool. These sub-MIPs are solved with HiGHS. To stabilize the LP convergence, the branch-and-price back end uses box-style dual stabilization [6] with weights 0.75, 0.50, and 0.25 and update weight 0.25. At the root node, the pricer is also called twice with a dual vector shifted halfway from the current dual solution toward the stabilized dual reference point. These additional root pricing calls add columns before they become negative at the current restricted master solution.

6.2 Lower Bound Track

The lower-bound track is the same as the exact track, except that the sub-MIP heuristic is disabled and the solver aborts early once the bound is sufficiently good.

6.3 Heuristic Track

The heuristic track uses the reductions from Section 2. Cluster-reduction subsolves use the same exact solver with a time limit of $\max\left(\frac{\#\text{cluster leaves}}{\#\text{total leaves}} \cdot 300, 0.25\right)$ seconds and lift the best incumbent found within that limit. If at most 1000 leaves remain after reductions, the heuristic track continues with the B&P solver and returns the best incumbent when time expires.

For larger reduced instances, the Lagrangian penalizers are optimized with the Conic Bundle subgradient optimizer [2]. The main column-generation phase uses at most 15% of the time remaining after reductions, the perturbation phase uses 5%, and both directions together use at most 40%. The restricted set-packing ILPs are solved over the best 3000, 7000, and 12000 generated components, with time limits of 10 seconds, 30 seconds, and the remaining solve time, respectively.

276 — References —

- 277 1 Martin Frohn, Steven Kelk, and Simona Vychytilova. A branch-&-price approach to the
 278 unrooted maximum agreement forest problem. *Oper. Res. Lett.*, 63:107364, 2025. URL:
 279 <https://doi.org/10.1016/j.orl.2025.107364>, doi:10.1016/J.ORL.2025.107364.
- 280 2 Christoph Helmberg. ConicBundle: A callable library for
 281 convex optimization, 2021. TU Chemnitz. Available at:
 282 https://www.tu-chemnitz.de/mathematik/discrete_math/helmberg/ConicBundle/.
- 283 3 Steven Kelk, Simone Linz, and Ruben Meuwese. Cyclic generators and an improved linear
 284 kernel for the rooted subtree prune and regraft distance. *Inf. Process. Lett.*, 180:106336, 2023.
 285 URL: <https://doi.org/10.1016/j.ipl.2022.106336>, doi:10.1016/J.IPL.2022.106336.
- 286 4 Steven Kelk, Ruben Meuwese, and Leo van Iersel. A kernel for the maximum agreement forest
 287 problem on multiple binary phylogenetic trees, 2026. URL: <https://arxiv.org/abs/2602.22281>,
 288 arXiv:2602.22281, doi:10.48550/arXiv.2602.22281.
- 289 5 Zhijiang Li and Norbert Zeh. Computing maximum agreement forests without cluster par-
 290 titioning is folly. In Kirk Pruhs and Christian Sohler, editors, *25th Annual European Sym-
 291 posium on Algorithms, ESA 2017, Vienna, Austria, September 4-6, 2017*, volume 87 of
 292 *LIPICs*, pages 56:1–56:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. URL:
 293 <https://doi.org/10.4230/LIPICs.ESA.2017.56>, doi:10.4230/LIPICs.ESA.2017.56.
- 294 6 Roy E. Marsten, W. W. Hogan, and J. W. Blankenship. The boxstep method for large-scale
 295 optimization. *Oper. Res.*, 23(3):389–405, 1975. URL: <https://doi.org/10.1287/opre.23.3.389>,
 296 doi:10.1287/OPRE.23.3.389.
- 297 7 Neil Olver, Frans Schalekamp, Suzanne van der Ster, Leen Stougie, and Anke van Zuylen. A
 298 duality based 2-approximation algorithm for maximum agreement forest. *Math. Program.*,
 299 198(1):811–853, 2023. URL: <https://doi.org/10.1007/s10107-022-01790-y>, doi:10.1007/
 300 S10107-022-01790-Y.
- 301 8 Feng Shi, Jianer Chen, Qilong Feng, Xiaojun Ding, and Jianxin Wang. Algorithms for
 302 maximum agreement forest of multiple general trees. *CoRR*, abs/1411.0062, 2014. URL:
 303 <http://arxiv.org/abs/1411.0062>, arXiv:1411.0062.

304 **A** Rooted Weighted Maximum Agreement Subtree Partition

305 The rooted weighted maximum agreement subtree partition (rWMASTP) problem asks for
 306 a partition $P \subseteq \mathcal{C}$ of $\mathcal{L}$ into compatible sets such that the Steiner trees of the blocks are
 307 internally disjoint in T_1 and the total weight is maximized. Given weights w_v on internal
 308 vertices $v \in \left(\bigcup_{t=1}^k V(T_t)\right) \setminus \mathcal{L}$ and weights w_e on edges $e \in \bigcup_{t=1}^k E(T_t)$, each component
 309 incurs a penalty of -1 . The weight of a compatible component $A \in \mathcal{C}$ is

$$310 \quad w(A) = -1 + \sum_{t=1}^k \left(\sum_{v \in \text{ST}(T_t, A) \setminus \mathcal{L}} w_v + \sum_{e \in E(\text{ST}(T_t, A))} w_e \right).$$

311 Note that the inclusion of weights on the internal vertices of the first tree T_1 is intentional,
 312 as we also use rWMASTP as a pricer in the B&P framework (detailed in Section 3.2). The
 313 edge weights w_e arise from dualizing branching decisions on edges when solving rWMASTP
 314 as a pricing subsolver in the B&P.

315 If we associate with each compatible component $A \in \mathcal{C}$ a binary variable x_A indicating

whether A is selected, we obtain the following integer programming formulation:

$$\begin{aligned}
 & \max \sum_{A \in \mathcal{C}} w(A) x_A \\
 & \text{s.t.} \quad \sum_{A \in \mathcal{C}: \ell \in A} x_A = 1 \quad \forall \ell \in \mathcal{L}, \\
 & \quad \sum_{A \in \mathcal{C}: v \in \text{ST}(T_1, A)} x_A \leq 1 \quad \forall v \in V(T_1) \setminus \mathcal{L}, \\
 & \quad x_A \in \{0, 1\} \quad \forall A \in \mathcal{C}.
 \end{aligned}$$

To solve the rWMASTP pricing subproblem efficiently on two trees T_1 and T_2 , we use a dynamic programming (DP) algorithm that sweeps bottom-up on both trees. The DP is inspired by the rooted weighted maximum agreement subtree (rWMAST) DP algorithm presented in [1]. For a vertex $u \in V(T)$, write $T|_u$ for the subtree rooted at u , and write $\mathcal{L}(T|_u)$ for the leaves below u . Instead of evaluating all exponential subset partitions, the DP restricts matching states to pairs of nodes $(u, v) \in V(T_1) \times V(T_2)$ whose leaf sets intersect:

$$\mathcal{S} = \{(u, v) \in V(T_1) \times V(T_2) \mid \mathcal{L}(T_1|_u) \cap \mathcal{L}(T_2|_v) \neq \emptyset\}.$$

For two trees, the size of this state space is bounded by $\mathcal{O}(n^2)$. For each state $(u, v) \in \mathcal{S}$, the DP computes two open state values representing the maximum total weight of a partition of the leaves $\mathcal{L}(T_1|_u)$ into compatible components, where all components are closed except for one active component $A \subseteq \mathcal{L}(T_1|_u)$ that is open at u in T_1 and at v in T_2 . Specifically, the active component's Steiner tree is extended to include the sweep-frontier nodes u and v , taking the form $\text{ST}(T_1, A \cup \{u\})$ in T_1 and $\text{ST}(T_2, A \cup \{v\})$ in T_2 . This connection to u and v makes it the only component in the partial partition that can still be extended upwards.

- $O(u, v)$: The maximum weight of a partition of $\mathcal{L}(T_1|_u)$ containing one such active component A , which is connected to u and v respectively.
- $O_{\text{close}}(u, v)$: The maximum weight of a partition of $\mathcal{L}(T_1|_u)$ where u and v are the least common ancestors of the active component A in T_1 and T_2 , respectively, making it eligible to be closed.

The DP also computes a closed forest value:

- $C(u)$: The maximum total weight of a partition of $\mathcal{L}(T_1|_u)$ where all components are closed.

The transitions propagate these values bottom-up: open states are initialized at matching leaves (ℓ, ℓ) , extended upwards along edges in both trees, combined at matching split nodes to transition to closable states, and closed at their least common ancestors to transition to the closed forest state $C(u)$. For each internal node $u \in V(T_1) \setminus \mathcal{L}$ with children u_L and u_R :

$$C(u) = \max \left\{ C(u_L) + C(u_R), -1 + \max_{v: (u, v) \in \mathcal{S}} O_{\text{close}}(u, v) \right\}.$$

where the first term represents splitting the forest (meaning u is not covered by any component), and the second term represents closing a component rooted at u with a component penalty of -1 . This sweep runs in $\mathcal{O}(n^2)$ time for two trees. For $k > 2$ trees, we can solve rWMASTP in $\mathcal{O}(kn^3)$ time. Here, states are mapped to the $\mathcal{O}(n^2)$ leaf pairs (a, b) , which uniquely define a vertex tuple $(u_1, \dots, u_k)$ of their least common ancestors across all trees. The DP evaluates $\mathcal{O}(n^3)$ transitions of the form $(a, b) \rightarrow (a, c)$, where c is a leaf such that $\text{lca}(T_t, \{a, c\})$ is a strict ancestor of $\text{lca}(T_t, \{a, b\})$ in all trees T_t .

Reductions and branching decisions can restrict the solution space by setting an edge to cut (meaning it cannot be part of the agreement forest). Cut edge restrictions are handled in the DP by disallowing the corresponding state transitions.